A/B TESTING ANALYSIS PROJECT IN PYTHON

1. Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import ttest_ind
```

Optional display settings

```
pd.set_option('display.max_columns', None)
sns.set_style("whitegrid")
```

2. Load the Datasets

```
control_df = pd.read_csv("control_group.csv", sep=';')
test_df = pd.read_csv("test_group.csv", sep=';')
```

3. Basic Inspection

```
print("========== CONTROL GROUP ==========")
print("Shape:", control_df.shape)
print(control_df.head())
print("\nData Types:\n", control_df.dtypes)
print("\nMissing Values:\n", control_df.isnull().sum())
print("\n========== TEST GROUP ==========")
print("Shape:", test_df.shape)
print(test_df.head())
print("\nData Types:\n", test_df.dtypes)
print("\nMissing Values:\n", test_df.isnull().sum())
```

4. Data Cleaning

Drop rows with missing values

```
control_df = control_df.dropna()
test_df = test_df.dropna()
```

Convert Date column to datetime format

```
control_df['Date'] = pd.to_datetime(control_df['Date'], format='%d.%m.%Y')
test_df['Date'] = pd.to_datetime(test_df['Date'], format='%d.%m.%Y')
```

Add campaign group label

```
control_df['Group'] = 'Control'
test_df['Group'] = 'Test'
```

5. Merge Both Datasets

```
ab_data = pd.concat([control_df, test_df], ignore_index=True)
print("\n========== MERGED DATA ==========")
print("Shape:", ab_data.shape)
print(ab_data.head())
```

6. Create KPI Metrics

CTR = Click Through Rate

```
ab_data['CTR'] = (ab_data['# of Website Clicks'] / ab_data['# of Impressions']) * 100
```

Conversion Rate = Purchases / Website Clicks

```
ab_data['Conversion Rate'] = (ab_data['# of Purchase'] / ab_data['# of Website Clicks']) * 100
```

CPC = Cost Per Click

```
ab_data['CPC'] = ab_data['Spend [USD]'] / ab_data['# of Website Clicks']
```

CPA = Cost Per Acquisition / Purchase

```
ab_data['CPA'] = ab_data['Spend [USD]'] / ab_data['# of Purchase']
```

Add to Cart Rate

```
ab_data['Add to Cart Rate'] = (ab_data['# of Add to Cart'] / ab_data['# of View Content']) * 100
```

Purchase from Cart Rate

```
ab_data['Purchase from Cart Rate'] = (ab_data['# of Purchase'] / ab_data['# of Add to Cart']) * 100
```

7. Descriptive Statistics

```
print("\n========== DESCRIPTIVE STATISTICS ==========")
print(ab_data.describe())
print("\n========== GROUP-WISE AVERAGE COMPARISON ==========")
group_avg = ab_data.groupby('Group').mean(numeric_only=True).round(2)
print(group_avg)
```

8. Compare Main Metrics

```
comparison = ab_data.groupby('Group')[[
    'Spend [USD]',
    '# of Impressions',
    'Reach',
    '# of Website Clicks',
    '# of Searches',
    '# of View Content',
    '# of Add to Cart',
    '# of Purchase'
]].mean().round(2)
print("\n========== AVERAGE PERFORMANCE COMPARISON ==========")
print(comparison)
```

9. Compare KPI Metrics

```
kpi_comparison = ab_data.groupby('Group')[[
    'CTR',
    'Conversion Rate',
    'CPC',
    'CPA',
    'Add to Cart Rate',
    'Purchase from Cart Rate'
]].mean().round(2)
print("\n========== KPI COMPARISON ==========")
print(kpi_comparison)
```

10. Visualization - Average Purchases

```
plt.figure(figsize=(8,5))
sns.barplot(data=ab_data, x='Group', y='# of Purchase', estimator=np.mean)
plt.title('Average Purchases by Campaign')
plt.xlabel('Campaign Group')
plt.ylabel('Average Purchases')
plt.show()
```

11. Visualization - Average Website Clicks

```
plt.figure(figsize=(8,5))
sns.barplot(data=ab_data, x='Group', y='# of Website Clicks', estimator=np.mean)
plt.title('Average Website Clicks by Campaign')
plt.xlabel('Campaign Group')
plt.ylabel('Average Website Clicks')
plt.show()
```

12. Visualization - Average Spend

```
plt.figure(figsize=(8,5))
sns.barplot(data=ab_data, x='Group', y='Spend [USD]', estimator=np.mean)
plt.title('Average Spend by Campaign')
plt.xlabel('Campaign Group')
plt.ylabel('Average Spend')
plt.show()
```

13. Visualization - CTR Comparison

```
plt.figure(figsize=(8,5))
sns.barplot(data=ab_data, x='Group', y='CTR', estimator=np.mean)
plt.title('Average CTR by Campaign')
plt.xlabel('Campaign Group')
plt.ylabel('CTR (%)')
plt.show()
```

14. Visualization - Conversion Rate Comparison

```
plt.figure(figsize=(8,5))
sns.barplot(data=ab_data, x='Group', y='Conversion Rate', estimator=np.mean)
plt.title('Average Conversion Rate by Campaign')
plt.xlabel('Campaign Group')
plt.ylabel('Conversion Rate (%)')
plt.show()
```

15. Daily Trend – Purchases

```
plt.figure(figsize=(12,6))
sns.lineplot(data=ab_data, x='Date', y='# of Purchase', hue='Group', marker='o')
plt.title('Daily Purchases Trend: Control vs Test')
plt.xlabel('Date')
plt.ylabel('Purchases')
plt.xticks(rotation=45)
plt.show()
```

16. Daily Trend - Website Clicks

```
plt.figure(figsize=(12,6))
sns.lineplot(data=ab_data, x='Date', y='# of Website Clicks', hue='Group', marker='o')
plt.title('Daily Website Clicks Trend: Control vs Test')
plt.xlabel('Date')
plt.ylabel('Website Clicks')
plt.xticks(rotation=45)
plt.show()
```

17. Correlation Heatmap

```
corr_data = ab_data.drop(columns=['Campaign Name', 'Date', 'Group'])
plt.figure(figsize=(14,10))
sns.heatmap(corr_data.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heatmap')
plt.show()
```

18. Hypothesis Testing – Purchases

Null Hypothesis (H0): No significant difference in purchases between Control and Test campaigns

Alternative Hypothesis (H1): Significant difference exists

```
control_purchases = ab_data[ab_data['Group'] == 'Control']['# of Purchase']

test_purchases = ab_data[ab_data['Group'] == 'Test']['# of Purchase']

t_stat, p_value = ttest_ind(control_purchases, test_purchases, equal_var=False)

print("\n========== T-TEST FOR PURCHASES ==========")

print("T-statistic:", round(t_stat, 4))

print("P-value:", round(p_value, 4))

alpha = 0.05

if p_value < alpha:

    print("Result: Reject Null Hypothesis → Significant difference in purchases between Control and Test.")

else:

    print("Result: Fail to Reject Null Hypothesis → No statistically significant difference in purchases.")
```

19. Function for A/B Testing on Multiple Metrics

```
def ab_test_metric(data, metric):

    control = data[data['Group'] == 'Control'][metric]

    test = data[data['Group'] == 'Test'][metric]

    t_stat, p_value = ttest_ind(control, test, equal_var=False)

    print(f"\nMetric: {metric}")

    print("T-statistic:", round(t_stat, 4))

    print("P-value:", round(p_value, 4))

    if p_value < 0.05:

        print("Result: Significant difference between Control and Test")

    else:

        print("Result: No statistically significant difference")
```

20. Run T-tests for Multiple Metrics

```
metrics = [
    '# of Website Clicks',
    '# of Add to Cart',
    '# of Purchase',
    'CTR',
    'Conversion Rate'
]
print("\n========== MULTIPLE METRIC A/B TEST RESULTS ==========")
for metric in metrics:
    ab_test_metric(ab_data, metric)
```

21. Final Summary Table

```
final_summary = ab_data.groupby('Group')[[
    'Spend [USD]',
    '# of Impressions',
    'Reach',
    '# of Website Clicks',
    '# of Searches',
    '# of View Content',
    '# of Add to Cart',
    '# of Purchase',
    'CTR',
    'Conversion Rate',
    'CPC',
    'CPA',
    'Add to Cart Rate',
    'Purchase from Cart Rate'
```

]].mean().round(2)

print("\n========== FINAL SUMMARY TABLE ==========")

print(final_summary)